CS452 HW3 – Unix Shell ReadMe

Caleb Corolewski

October 15, 2026

1 Introduction

In this assignment, we implemented a simplified Unix shell in C. The shell reads a command line, parses it into commands and operators, and executes the requested operations. External programs are launched using `fork()` and `execvp()`. The parent shell waits for foreground processes, while background processes can run without blocking the next prompt.

The shell supports multiple commands separated by semicolons (`;`), background execution using `&`, pipelines using `|`, and input and output redirection using `<` and `>`. It also supports the built-in commands `pwd`, `cd`, `cd -`, `history`, and `exit`. Built-ins that must change shell state, such as `cd`, are executed in the parent shell when used alone in the foreground. Commands in pipelines are executed in child processes.

2 File Contents

The main source files for HW3 are organized as follows:

```
hw3/
├── GNUmakefile
├── Command.c
├── Command.h
├── Interpreter.c
├── Interpreter.h
├── Jobs.c
├── Jobs.h
├── Parser.c
├── Parser.h
├── Pipeline.c
├── Pipeline.h
├── Scanner.c
├── Scanner.h
├── Sequence.c
├── Sequence.h
├── Shell.c
├── Tree.c
├── Tree.h
├── deq.c
├── deq.h
├── error.h
├── run
└── Test/
    ├── Test_01_echo/
    ├── Test_02_multiple_lines/
    ├── ...
    ├── Test_15_pipe_and_sequence/
    └── Test_pwd/
```

The `Command` module represents individual commands and handles argument lists, redirection, built-ins, and external program execution. The `Pipeline` module connects commands with pipes and manages their processes. The `Jobs` module tracks background process IDs and reaps completed processes. The `Sequence` module executes groups of pipelines. The `Scanner`, `Parser`, and `Tree` modules provide the command-line syntax representation, and `Interpreter` translates that representation into executable commands. `Shell.c` contains the main read–execute loop. The deque implementation in `deq.c` supports the shell's command and pipeline collections.

3 Building and Running

On Ubuntu, the GNU Readline development package is needed to compile the shell. It can be installed with:

```
sudo apt install libreadline-dev
```

From the HW3 directory, compile with:

```
make
```

The build produces an executable named `shell`. Start it interactively with:

```
./shell
```

Alternatively, provide commands using standard input:

```
printf 'echo hello\nexit\n' | ./shell
```

Example interactive commands include:

```
pwd
cd /etc
cd -
echo hello | wc -c
echo hello > output.txt
cat < output.txt
sleep 5 &
history
exit
```

Operators should be separated from words by whitespace to match the provided scanner's expected input format.

4 Testing

A set of 15 additional automated tests was prepared for the shell. The tests cover basic external commands, multiple input lines, command sequences, `pwd`, `cd`, two-stage and three-stage pipelines, output redirection, input redirection, combined redirection, background execution, and the behavior of `exit`. A test also checks a pipeline followed by another command. The supplied `Test_pwd` test is retained.

Each test directory contains an `inp` file holding commands for the shell and an `exp` file containing the expected standard output. The `run` script executes each input and creates or updates an `out` file for comparison. To execute the suite from the HW3 directory, run:

```
chmod +x run
./run shell
```

To inspect a particular test failure, compare its expected and actual output:

```
diff -u Test/Test_07_pipe_count/exp \
        Test/Test_07_pipe_count/out
```

The `pwd` test requires special attention because an expected absolute directory path from another computer will not match the working directory on the current computer. When running the suite from the HW3 directory, its expected output can be updated with:

```
pwd > Test/Test_pwd/exp
```

These tests check the specified output and command behavior. Background execution should also be checked interactively to confirm that the prompt returns without waiting for the background process. Memory management can be checked with Valgrind. Passing results should be reported only after running the suite against the final version of the shell.